

Playwright Automation Framework Guide

Production-oriented keyword-driven framework documentation

Playwright

Keyword Driven

Excel Based

PDF Validation

SUPPORTED FIELD TYPES

12

ACTION KEYWORDS

10+

CORE LAYERS

3

DATA SOURCES

Excel + .env

Index

1. Introduction
2. Framework Capabilities
3. Architecture Diagram
4. Execution Flow
5. Field Types and Actions
6. TestSuite Structure
7. ObjectRepository Structure
8. Example Data Sheets
9. Environment Configuration
10. Extensibility Guide for Testers
11. Documentation Standards

1) Introduction

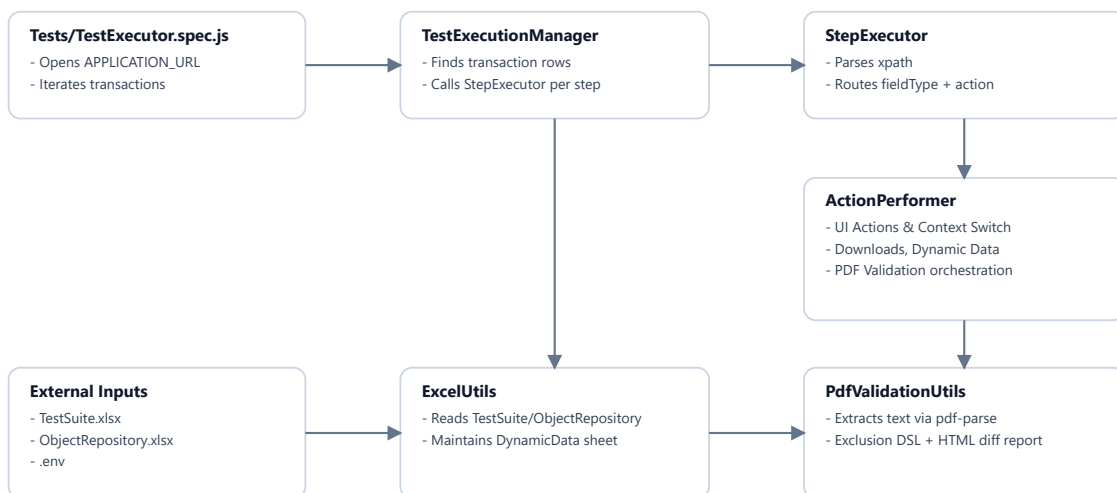
This framework is a **keyword-driven Playwright automation framework** where test steps are authored in Excel sheets and executed through reusable action handlers.

It is designed for both automation engineers and business users by separating **test intent (what to do)** from **technical implementation (how to do)**.

2) Framework Capabilities

- Data-driven transaction execution from TestSuite and ObjectRepository.
- Supports common UI actions (input, click, validation, dropdown, window/frame switch).
- Dynamic data read/write support for cross-step data management.
- Download handling and advanced PDF comparison with HTML diff reports.
- Config-driven behavior using .env (URLs, paths, report settings, exclusions).

3) Architecture Diagram



Legend: The framework takes data from Excel + .env, routes steps through execution layers, and generates actionable reports.

4) Execution Flow (Standard Procedure)

1. Read test cases from TestSuite control sheet and filter executable rows.
2. Open browser using APPLICATION_URL.
3. Resolve each transaction to corresponding TestSuite and ObjectRepository sheets.
4. For each step, map fieldType + action to the exact performer method.

5. Execute UI/API/data actions; generate Playwright + custom reports for evidence.

5) Field Types and Associated Actions

Text Box

ACTION	DESCRIPTION	TEST DATA USAGE
I	Inputs text into a textbox locator.	Enter the text value.

Button

ACTION	DESCRIPTION	TEST DATA USAGE
C	Clicks a button or clickable element.	Any non-null value to trigger action.

Checkbox

ACTION	DESCRIPTION	TEST DATA USAGE
C	Checks/Unchecks based on test data.	Use exactly Check or Uncheck.

Label

ACTION	DESCRIPTION	TEST DATA USAGE
V	Validates locator text equals expected.	Expected visible text.

Dropdown

ACTION	DESCRIPTION	TEST DATA USAGE
I	Selects by option value.	Option value/string.
INDEX_SELECTION	Selects dropdown item by index.	Numeric index (single digit in current logic).

SwitchToIframe

ACTION	DESCRIPTION	TEST DATA USAGE
SWITCH	Switches context into iframe.	Any non-null trigger value.

SwitchToWindow

ACTION	DESCRIPTION	TEST DATA USAGE
SWITCH_WINDOW	Clicks locator and switches to newly opened window/tab.	Any non-null trigger value.

SwitchToMainWindow

ACTION	DESCRIPTION	TEST DATA USAGE
SWITCH	Switches back to parent/main page context.	Not required.
SWITCH_BY_TITLE	Switches to window by title.	Provide target window title.

DynamicDataHandler

ACTION	DESCRIPTION	TEST DATA USAGE
W; <COLUMN_NAME>	Writes extracted UI text to dynamic data sheet.	Uses current locator text content.

PdfValidation

ACTION	DESCRIPTION	TEST DATA USAGE
V	Downloads PDF, compares with expected PDF, applies exclusion DSL, generates HTML diff report and attaches artifacts.	Expected file name (with/without .pdf).

API

ACTION	DESCRIPTION	STATUS
GET	Reserved for API GET execution path.	Placeholder

OCR

ACTION	DESCRIPTION	STATUS
N/A	OCR field type hook exists for future implementation.	Placeholder

6) TestSuite Structure

Control Sheet: Used to list scenarios, execution flag, and transactions.

COLUMN	MEANING
A	TestCaseName
B	Category
C	Execute (Y/N)
D onward	Transaction names to run in sequence

Transaction Sheets: Row 1 = TestCase IDs across columns, Row 2 = Transaction names, Row 3 onward = step data with Field Name in column A.

7) ObjectRepository Structure

COLUMN	MEANING
A	Field Name (must match TestSuite field name)
B	Field Type (e.g., Text Box, PdfValidation)
C	Action keyword (e.g., I, C, V, SWITCH)
D	Xpath / locator expression

8) Sample Data Structures

Sample TestSuite (Transaction Sheet)

FIELD NAME	TC_PDF_001 / TXN_PDF_VALIDATE
Download Brochure	Get_Started_With_Smallpdf.pdf
Customer Name	John Doe
Invoice Date	12/04/2026

Sample ObjectRepository (Same Sheet Name)

FIELD NAME	FIELD TYPE	ACTION	XPATH
Download Brochure	PdfValidation	V	//a[@id='download-pdf']
Customer Name	Text Box	I	//input[@name='customerName']
Invoice Date	Label	V	//span[@id='invoiceDate']

9) Environment Configuration (.env)

VARIABLE	PURPOSE	EXAMPLE
APPLICATION_URL	Application base URL for test navigation	https://example.com
DOWNLOAD_FOLDER_PATH	Target folder for downloaded PDFs and reports	F:/PlaywrightTests/AutomationFramework/project/test/downl
EXCLUDE_CONTENT	Custom DSL for dynamic text exclusions in PDF comparison	WORD:John;;DATE:ANY;;REGEX:/Invoice\\s*No\\s*:\\s*\\w+/i

Tip Keep expected files under DOWNLOAD_FOLDER_PATH/expected.

10) Extensibility Guide for Automation Testers

This framework follows a **dispatch pattern** where each step is mapped by `fieldType` + `action`. To extend the framework safely, add code in a controlled sequence:

ActionPerformer method → **StepExecutor route** → **Excel updates**.

A) Add New Action to Existing Field Type

Use case: Add action `DBL_C` (double click) for existing `Button` field type.

1. Create a reusable method in `ActionPerformer.js`.
2. Add a new route condition in `StepExecutor.js`.
3. Update `ObjectRepository` row with field type and new action keyword.
4. Add test data row in `TestSuite` transaction sheet.

```
/* ActionPerformer.js */
async performDoubleClickAction(testData, locator){
  if(testData !== null){
    await this.page.locator(locator).dblclick();
  }
}

/* StepExecutor.js */
else if(objectRepositoryStep.fieldType==="Button" && objectRepositoryStep.action==="DBL_C"){
  await this.actionPerformer.performDoubleClickAction(testSuiteStep.data, objectRepositoryStep.x
}
```

Technical recommendation: keep action methods **idempotent** where possible and always include clear guard clauses for null/invalid test data.

B) Add a Brand-New Field Type

Use case: Add `FileUpload` field type with action `u`.

1. Define a strongly scoped method in `ActionPerformer.js`.
2. Register new dispatch branch in `StepExecutor.js`.
3. Add `ObjectRepository` row with `FieldType=FileUpload`, `Action=u`, locator for file input.
4. Provide file path in `TestSuite` data cell.

```
/* ActionPerformer.js */
import fs from "fs";

async performFileUploadAction(testData, locator){
  if(testData !== null){
    if(!fs.existsSync(testData)){
      throw new Error(`Upload file not found: ${testData}`);
    }
    await this.page.locator(locator).setInputFiles(testData);
  }
}

/* StepExecutor.js */
else if(objectRepositoryStep.fieldType==="FileUpload" && objectRepositoryStep.action==="U"){
  await this.actionPerformer.performFileUploadAction(testSuiteStep.data, objectRepositoryStep.x
}
```

C) Sample Excel Rows for New Implementations

FIELD NAME	FIELD TYPE	ACTION	XPATH	TESTSUITE DATA EXAMPLE
Submit Request	Button	DBL_C	//button[@id='submit']	Y
Upload Invoice	FileUpload	U	//input[@type='file']	F:/test-data/invoice.pdf

D) Engineering Checklist (Production-Grade)

- Use deterministic action keywords and avoid overloaded semantics.
- Validate preconditions (path existence, locator validity, required env variables).
- Emit meaningful failures with step context (`fieldName`, `fieldType`, `action`).
- Attach artifacts when behavior is complex (downloaded files, diff reports, screenshots).
- Prefer reusable utility classes for cross-cutting logic (parsing, comparison, transformations).

11) Documentation & Usage Standards

- Use consistent naming between TestSuite Field Name and ObjectRepository Field Name.
- Use explicit action keywords and valid field types only.
- Store expected PDFs in `expected` folder with stable naming convention.
- For dynamic documents, use `EXCLUDE_CONTENT` DSL to avoid false mismatches.
- Attach generated reports as part of execution evidence in Playwright HTML report.